EXPERIMENT 8

Student Name: Kapish Gupta **UID: 24BAI70577**
Branch: AIT-CSE (AIML) **Section/Group: 24AIT_KRG-G2**
Semester: 4
Subject Name: Database Management System **Subject Code: 24CSH-298**

EXPERIMENT 7

Design of a Parameterized PL/SQL Stored Procedure for Employee Count by Gender (CData, Toddle)

AIM

To design and implement a stored procedure that accepts gender as an input parameter and returns the total number of employees based on the given gender, demonstrating procedural programming and parameterized logic in PL/SQL.

OBJECTIVE

- **To write and execute a stored procedure in PL/SQL**
- **To dynamically accept gender as an input parameter**
- **To compute and return employee count based on the given gender**

SOFTWARE REQUIREMENTS

Database Management System:
Oracle Database Express Edition (Oracle XE)
PostgreSQL Database

Database Administration Tool / Client Tool:
Oracle SQL Developer (for Oracle XE)
pgAdmin (for PostgreSQL)

EXPERIMENT 8

PROBLEM STATEMENT

Organizations require flexible reporting systems to analyze workforce distribution based on gender for:

- **Compliance requirements**
- **Internal analytics**
- **Decision-making**

PRACTICAL / EXPERIMENT STEPS

- ☐ **Create an employee table (if not already present)**
- ☐ **Insert sample employee data with gender field**
- ☐ **Design a stored procedure that:**
  - **Accepts gender as an input parameter**
  - **Counts employees matching that gender**
  - **Returns the result using an output parameter**
- ☐ **Execute the procedure with different gender inputs**
- ☐ **Display the output**

STEP-WISE OUTPUT

```sql
CREATE TABLE employees (
    emp_id SERIAL PRIMARY KEY,
    emp_name VARCHAR(50),
    gender VARCHAR(10)
);

INSERT INTO employees (emp_name, gender) VALUES
('Aman', 'Male'),
('Riya', 'Female'),
('John', 'Male'),
('Sara', 'Female'),
('Neha', 'Female'),
('Rahul', 'Male');
```

EXPERIMENT 8

```
CREATE OR REPLACE FUNCTION get_employee_count_by_gender(p_gender VARCHAR)
RETURNS INTEGER
LANGUAGE plpgsql
AS $$
DECLARE
    emp_count INTEGER;
BEGIN
    SELECT COUNT(*) INTO emp_count
    FROM employees
    WHERE gender = p_gender;

    RETURN emp_count;
END;
$$;

SELECT get_employee_count_by_gender('Male') AS male_count;
SELECT get_employee_count_by_gender('Female') AS female_count;
```

Data Output Messages Notifications

	female_count integer
1	3

RESULT ANALYSIS

1. **Simple View**
   - **Created from a single table.**
   - **Executes quickly for small datasets.**
   - **No performance optimization for complex operations.**

2. **Complex View**
   - **Includes joins and aggregation.**
   - **Slower than simple view because data is computed every time the query runs.**
3. **Materialized View**
   - **Stores precomputed results physically.**
   - **Significantly faster for repeated queries.**
   - **Requires refresh mechanism to maintain updated data.**

REAL-WORLD APPLICATION

Companies such as SanDisk, JTG, and PayPal handle large-scale transactional data.

In such organizations:

- **Complex reports run frequently.**
- **Data warehouses require fast query responses.**
- **Business intelligence dashboards need optimized performance.**

Materialized views are used to:

- **Improve reporting speed.**
- **Reduce server workload.**
- **Enhance query optimization.**
- **Improve overall system performance.**

LEARNING OUTCOME

After completing this experiment, the learner is able to:

After completing this experiment, the learner will be able to:

- **Understand the structure of a stored procedure**
- **Use IN and OUT parameters effectively**

EXPERIMENT 8

- **Pass dynamic inputs to procedures**
- **Apply aggregate functions (COUNT) in PL/SQL**
- **Develop reusable database logic for real-world applications (e.g., CData, Toddle)**

CONCLUSION

This experiment demonstrates how PL/SQL stored procedures can be used to create dynamic and reusable database logic. By passing parameters and using aggregate functions, organizations can efficiently generate reports and insights based on specific conditions like gender.